Kanbien packages/core Architecture Guide

Version 2 - core contracts, platform adapters, and vendor strategy

Updated to include provider-specific platform folders, Option B package-per-adapter structure, and AWS / Azure / open-source provider recommendations.

Prepared for greenfield platform design

Date: 22 June 2026

The main architectural decision in this version is that **packages/core** remains provider-neutral, while **platform/** is organized by provider and uses **one package per adapter** for long-term scalability.

Table of contents

Note: all H1 sections start on a new page. Page numbers are in the footer.

Revision summary

This edition incorporates the later design discussions about platform vendor separation and makes a specific decision: Kanbien should use Option B, meaning one package per adapter, under provider-specific folders.

What changed from the first guide

- Added a formal architecture decision for provider-specific platform folders and package-per-adapter implementation packages.
- Clarified that packages/core contains Kanbien concepts, types, ports, errors, and policies - not AWS, Azure, or open-source vendor clients.
- Introduced platform/shared for provider-neutral adapter helpers and platform/local for local/test implementations.
- Added an AWS, Azure, and open-source provider recommendation for each core capability where an industry-standard option exists.
- Updated the final repository structure so platform/aws, platform/azure, platform/oss, and platform/local can evolve independently.
- Preserved the long-term principle that apps should consume core interfaces while deployment bootstrap code wires the selected platform adapters.

The key decision

Kanbien will start AWS-first, but the repository should be structured so future Azure, open-source, self-hosted, or enterprise-private deployments can be added without rewriting product logic. The chosen pattern is provider-first, adapter-per-package: for example, platform/aws/files-s3 and platform/azure/files-blob are separate packages that implement the same core FileStorage port.

The most important rule

```
packages/core = Kanbien contracts and concepts
platform/shared = provider-neutral adapter implementation helpers
platform/aws/* = AWS implementations of core contracts
platform/azure/* = Azure implementations of core contracts, when needed
platform/oss/* = open-source/self-hosted implementations, when needed
platform/local/* = local development and test implementations
apps/* = product logic and composition/bootstrap
```

Part 1 - Foundations: what packages/core is

Shared core package

A shared core package is an internal library that defines stable concepts and contracts used across the platform. In Kanbien, packages/core should define the common language for identity, permissions, tenancy, events, jobs, validation, logging, monitoring, audit, files, notifications, configuration, localization, and similar cross-cutting concerns.

Think of it like the building code and utility standards for a city. It does not contain every building, every business, or every street. It defines the rules and standard interfaces that let the city grow without chaos.

Why shared core packages exist

Without a shared core layer, every app invents its own answer to common questions: who is the user, which tenant are they in, what are they allowed to do, how should events look, how are files stored, how are audit records written, and how are errors reported. At small scale this creates annoyance. At enterprise scale it creates risk.

What core solves

- Consistency - the same Principal, TenantContext, Permission, EventEnvelope, AuditEvent, and ValidationError types are used everywhere.
- Reuse - apps do not rebuild validation, logging, authn, authz, audit, events, or file abstractions from scratch.
- Governance - security, audit, tenant isolation, redaction, retention, and event versioning can be standardized.
- Developer speed - new apps and workers can start from a known foundation.
- Future optionality - platform vendors can be swapped by replacing adapters, not product logic.

What core can break when misused

- It becomes a dumping ground for anything shared twice.
- It creates hidden coupling when core imports app, platform, or infrastructure code.
- It encourages over-abstraction before product needs are real.
- It increases blast radius because bugs in core can affect every app.
- It becomes a bottleneck when every small change requires coordination across all teams.

How core differs from adjacent layers

Concept	What it is	Relationship to packages/core
Applications	Deployable products and services such as API, admin app, portal, and workers.	Apps consume core. Core must not import apps.
Features	Product capabilities such as onboarding, billing, search, workflows, or reporting screens.	Features may use core primitives, but feature workflows do not belong in core.
Platform	Concrete runtime implementations of core ports, such as S3 FileStorage or SQS Queue.	Platform implements core contracts. Core does not know platform exists.
Infrastructure	Provisioned cloud resources: buckets, queues, databases, IAM roles, networks, alarms.	Infrastructure creates resources. Platform code uses them. Core only defines contracts.
Design system	Shared UI components, tokens, form controls, layout, accessibility, and display components.	Design system can consume i18n/localization and validation concepts from core.
Utility libraries	Generic helper functions such as date, string, array, or object helpers.	Utilities are not automatically core. Core should contain Kanbien meaning, not random helpers.

Admission rule for core

> A module belongs in packages/core only if it defines a stable, cross-cutting Kanbien concept or contract that multiple parts of the system must use consistently. Reuse alone is not enough.

Part 2 - Kanbien architecture stack and dependency direction

Repository shape

```
kanbien/
apps/ deployable products and services
packages/ shared libraries, including packages/core
platform/ runtime implementations of core contracts
infra/ cloud and infrastructure-as-code resources
harness/ local/dev/test orchestration
tools/ build, release, codegen, migration tooling
docs/ architecture, operations, and decision records
```

Conceptual stack

```
Apps
->
Packages
->
Platform
->
Infrastructure
```

Conceptually, apps use packages; packages define reusable concepts; platform implements the runtime capabilities; infrastructure provisions the actual resources. In code, the most important rule is that packages/core must not import platform, infra, or apps.

Recommended code dependency direction

```
apps/features -> packages/core
apps/bootstrap -> platform/<provider>/<adapter-package>
platform/<provider> -> packages/core
platform/shared -> packages/core, only for shared implementation helpers
infra -> no runtime imports from apps or core

packages/core -X-> apps
packages/core -X-> platform
packages/core -X-> infra
```

Why direction matters

The stable layer should not depend on the volatile layer. Product workflows will change often. Core contracts should change carefully. If core imports app code, then changing an app can break the foundation under every other app.

Architectural decay pattern

```
1. A feature needs a helper and puts it in core.
2. The helper imports an app type.
3. Another app depends on that helper.
4. The app type becomes a platform-wide dependency.
5. Nobody can change it without breaking unrelated systems.
```

> The correction is simple but strict: apps depend on core; platform implements core; infra provisions resources; core knows nothing about apps, platform, or infra.

Part 3 - Architecture decision: provider folders and one package per adapter

ADR-001: Option B is accepted

Kanbien should use Option B: one package per adapter. The provider folder groups adapters by deployment provider, and each adapter is its own package with its own dependencies, tests, README, versioning boundary, and implementation code.

```
platform/
shared/
local/
events-in-memory/
queues-in-memory/
files-local/
aws/
files-s3/
queues-sqs/
events-eventbridge/
notifications-ses/
authn-cognito/
azure/ # future
files-blob/
queues-service-bus/
events-event-grid/
oss/ # future
files-minio/
queues-rabbitmq/
events-nats/
```

Why provider-first

Provider-first structure reflects how real deployments are usually operated. A customer or environment is often AWS-first, Azure-first, or self-hosted. Security, IAM, networking, observability, deployment, and incident response are provider-shaped concerns.

Why one package per adapter

- Dependency isolation - the S3 adapter can depend on the AWS S3 SDK without pulling that dependency into every AWS adapter.
- Independent testing - each adapter can have contract tests and provider-specific integration tests.
- Independent evolution - queues, files, events, authn, and notifications can mature at different speeds.
- Clear ownership - teams can own adapter packages without owning the whole provider platform.
- Cleaner future portability - Azure and OSS adapters can be added without disturbing AWS packages.
- Lower blast radius - a change to SES notifications need not affect S3 files or SQS queues.

The cost of Option B

- More packages and more boilerplate.
- More dependency and version management.
- More README and testing discipline required.
- More initial repository structure than a single platform/aws package.

Those costs are real. But because Kanbien is being designed for long-term enterprise scalability, the package-per-adapter model is a reasonable upfront investment. The trick is not to implement unused Azure or OSS packages now. Define the pattern; implement only the packages you actually use.

Package naming convention

```
@kanbien/platform-aws-files-s3
@kanbien/platform-aws-queues-sqs
@kanbien/platform-aws-events-eventbridge
@kanbien/platform-aws-notifications-ses
@kanbien/platform-azure-files-blob
@kanbien/platform-oss-files-minio
@kanbien/platform-local-events-in-memory
```

Adapter package anatomy

```
platform/aws/files-s3/
package.json
README.md
src/
index.ts
s3-file-storage.ts
s3-signed-url-service.ts
config.ts
errors.ts
test/
contract.test.ts
integration.test.ts
```

Example adapter package.json

```
{
"name": "@kanbien/platform-aws-files-s3",
"version": "0.1.0",
"private": true,
"main": "dist/index.js",
"types": "dist/index.d.ts",
"dependencies": {
"@aws-sdk/client-s3": "^3.x"
},
"peerDependencies": {
"@kanbien/core": "workspace:*"
}
}
```

Example adapter implementation

```
import type { FileStorage, PutFileInput, StoredFile } from "@kanbien/core/files";

export interface S3FileStorageConfig {
bucketName: string;
region: string;
}

export class S3FileStorage implements FileStorage {
constructor(private readonly config: S3FileStorageConfig) {}

async put(input: PutFileInput): Promise<StoredFile> {
// AWS SDK call lives here, not in packages/core.
// Convert Kanbien FileStorage input into an S3 PutObject request.
return {
id: "file_123",
filename: input.filename,
contentType: input.contentType,
sizeBytes: input.sizeBytes,
tenantId: input.tenantId,
createdAt: new Date(),
};
}
}
```

Example app bootstrap

```
import { S3FileStorage } from "@kanbien/platform-aws-files-s3";
import { SqsQueue } from "@kanbien/platform-aws-queues-sqs";
import { EventBridgeEventBus } from "@kanbien/platform-aws-events-eventbridge";

export function createPlatform(config: RuntimeConfig) {
return {
files: new S3FileStorage({
bucketName: config.filesBucketName,
region: config.awsRegion,
}),
queue: new SqsQueue({
queueUrl: config.queueUrl,
region: config.awsRegion,
}),
eventBus: new EventBridgeEventBus({
busName: config.eventBusName,
region: config.awsRegion,
}),
};
}
```

Feature code should import only @kanbien/core types. Bootstrap code may import platform adapter packages because bootstrap is where a deployment chooses AWS, Azure, local, or open-source implementations.

Part 4 - Updated packages/core design rules

Core is a contract layer, not a vendor layer

Each packages/core module should expose types, interfaces, errors, policies, and small pure utilities. It may include test fakes where useful, but concrete integrations belong under platform/.

Recommended module shape

```
packages/core/src/<module>/
index.ts public exports
types.ts stable shared types
ports.ts interfaces implemented by platform adapters
errors.ts module-specific errors
policy.ts small pure policy helpers, when appropriate
testing.ts in-memory/no-op fakes, when useful
```

Core must not contain

- AWS, Azure, or vendor SDK clients.
- Terraform, Pulumi, CDK, Bicep, or other infrastructure definitions.
- Product feature workflows.
- Concrete database queries tied to one app's schema.
- Provider-specific environment variable names except generic config keys owned by Kanbien.
- UI components or design-system implementation details.

Updated packages/core folder structure

```
packages/core/
src/
shared/
config/
logging/
monitoring/
security/
authn/
authz/
tenancy/
validation/
persistence/
events/
queues/
async-jobs/
notifications/
files/
audit/
reporting/
analytics/
i18n/
localization/
testing/
```

Core to platform relationship

Core module	Core defines	Platform adapter implements
files	FileStorage interface and file metadata types	S3, Azure Blob, MinIO, local filesystem
queues	Queue and message envelope interface	SQS, Service Bus, RabbitMQ, NATS, in-memory
events	DomainEvent, EventEnvelope, EventBus	EventBridge, Event Grid, NATS, Kafka, in-memory
authn	Principal and Authenticator	Cognito, Entra External ID, Keycloak, fake auth
authz	Authorizer and policy decision model	Verified Permissions, Entra app roles, OpenFGA, OPA, simple RBAC
notifications	NotificationService and delivery types	SES, Azure Communication Services, Novu, console
monitoring	Metrics and health-check ports	CloudWatch, Azure Monitor, Prometheus/OpenTelemetry

Part 5 - Core module recommendations and provider adapter map

This section updates every core module with a clear recommendation for provider integration. The provider columns are not instructions to implement everything now. They define safe future paths and package names.

Summary matrix - identity, security, governance

Module	AWS	Azure	Open-source	Kanbien recommendation
authn	Cognito User Pools	Entra External ID	Keycloak	Use managed identity for v1. Keep Principal in core.
authz	Verified Permissions	Entra app roles/groups	OpenFGA or OPA	Start with simple Kanbien RBAC. Add fine-grained engine only when product complexity demands it.
security	KMS, Secrets Manager, WAF	Key Vault, WAF, Defender for Cloud	Vault, OPA, OWASP ZAP	Core owns safe primitives and interfaces; providers own key, secret, and edge protection implementations.
tenancy	Organizations/accounts for hard isolation	Management groups/subscriptions	Postgres RLS, namespaces	Tenancy is a Kanbien model first. Cloud provider isolation is an implementation choice.
audit	App audit plus CloudTrail/S3 Object Lock	App audit plus Activity Log/immutable Blob	Postgres audit, pgAudit, immudb	Kanbien must own application audit. Cloud audit is not enough.

Summary matrix - data, storage, reporting, analytics

Module	AWS	Azure	Open-source	Kanbien recommendation
persistence	Aurora PostgreSQL or RDS PostgreSQL	Azure Database for PostgreSQL Flexible Server	PostgreSQL	Use PostgreSQL as the default system of record.
files	S3	Azure Blob Storage	MinIO	Use object storage behind FileStorage.
reporting	QuickSight later	Power BI Embedded later	Superset or Metabase	Start with app-generated CSV/PDF reports; add BI when demand is real.
analytics	Firehose, S3, Redshift	Event Hubs, Fabric, Power BI	PostHog, ClickHouse	Use PostHog for early product analytics; warehouse later.

Summary matrix - events, queues, jobs, notifications

Module	AWS	Azure	Open-source	Kanbien recommendation
events	EventBridge	Event Grid	NATS JetStream or Kafka	Use outbox plus simple event bus first; add provider bus when multiple consumers emerge.
queues	SQS	Service Bus	RabbitMQ or NATS JetStream	Use queues for async work before introducing complex workflow engines.
async-jobs	SQS workers, Step Functions, EventBridge Scheduler	Service Bus, Functions, Durable Functions	Temporal, Kubernetes CronJobs	Start with queue-backed workers and scheduled jobs.
notifications	SES, End User Messaging where needed	Azure Communication Services	Novu, Postal	Use SES/ACS for delivery; use Novu only if orchestration is needed.

Summary matrix - runtime quality and global readiness

Module	AWS	Azure	Open-source	Kanbien recommendation
validation	API Gateway validation	API Management validation	Zod or Ajv	Use Zod in app/core-facing code; gateway validation is defense in depth.
config	AppConfig, Parameter Store, Secrets Manager	App Configuration, Key Vault	OpenFeature, Unleash, Flagsmith, Vault	Centralize typed config and secret references from day one.
logging	CloudWatch Logs	Azure Monitor Logs	OpenTelemetry + Loki/ELK	Use structured JSON logs and correlation IDs from day one.
monitoring	CloudWatch	Azure Monitor + Application Insights	Prometheus + Grafana + OTel	Expose metrics and health checks through core interfaces.
i18n	Amazon Translate	Azure Translator	i18next	Use i18next-style catalogs for product copy; machine translation is assistive, not authoritative.
localization	No strong AWS default	No strong Azure default	Intl, FormatJS, CLDR	Use standards for date, currency, number, and timezone formatting.

Detailed module recommendations

async-jobs/

Purpose. Background and scheduled work that should not block a user request.

Layer	Responsibility
packages/core	Job, JobHandler, JobScheduler, retry policy, idempotency key, job status, scheduling contract.
apps	Specific job types and handlers, such as sendWelcomeEmail or generateTenantUsageReport.
platform	Queue-backed scheduler, worker runtime, cron/schedule provider, retry/DLQ operations.

Provider	Recommended adapter package
AWS	@kanbien/platform-aws-jobs-sqs, @kanbien/platform-aws-workflows-step-functions, @kanbien/platform-aws-scheduler-eventbridge
Azure	@kanbien/platform-azure-jobs-service-bus, @kanbien/platform-azure-workflows-durable-functions, @kanbien/platform-azure-scheduler-functions-timer
Open-source/local	@kanbien/platform-oss-workflows-temporal, @kanbien/platform-oss-scheduler-kubernetes-cronjobs

Kanbien v1 recommendation. Use SQS-backed workers on AWS. Keep Step Functions or Temporal for later complex, stateful, long-running workflows.

authn/

Purpose. Authentication: proving who the caller is.

Layer	Responsibility
packages/core	Principal, Authenticator, authentication input, session/token errors.
apps	Login routing, onboarding, account linking, user profile flows.
platform	JWT verification, session verification, IdP integration, service identity verification.

Provider	Recommended adapter package
AWS	@kanbien/platform-aws-authn-cognito
Azure	@kanbien/platform-azure-authn-entra-external-id
Open-source/local	@kanbien/platform-oss-authn-keycloak

Kanbien v1 recommendation. Use managed authn. For AWS-first, Cognito is the native default. Keep Principal in core so provider choice does not leak into product logic.

authz/

Purpose. Authorization: deciding what an authenticated principal may do.

Layer	Responsibility
packages/core	AuthorizationRequest, AuthorizationDecision, Authorizer, Permission, PolicyError.
apps	Resource-specific permissions and business rules.
platform	External policy engine adapters and role/group claim mapping.

Provider	Recommended adapter package
AWS	@kanbien/platform-aws-authz-verified-permissions
Azure	@kanbien/platform-azure-authz-entra-app-roles
Open-source/local	@kanbien/platform-oss-authz-openfga, @kanbien/platform-oss-authz-opa

Kanbien v1 recommendation. Use simple RBAC inside Kanbien first. Add Verified Permissions/OpenFGA/OPA only when resource sharing and delegated administration become complex.

notifications/

Purpose. Messages sent to users or systems: email, SMS, push, in-app, webhook.

Layer	Responsibility
packages/core	Notification, NotificationRecipient, NotificationService, delivery status, template reference, preference contract.
apps	When to notify, who to notify, template content, escalation workflows.
platform	Provider delivery, template rendering, bounce handling, delivery status updates.

Provider	Recommended adapter package
AWS	@kanbien/platform-aws-notifications-ses, @kanbien/platform-aws-notifications-end-user-messaging
Azure	@kanbien/platform-azure-notifications-communication-services
Open-source/local	@kanbien/platform-oss-notifications-novu, @kanbien/platform-oss-email-postal

Kanbien v1 recommendation. Use SES for transactional email on AWS. Avoid building a full notification orchestration layer until channels and preferences justify it.

validation/

Purpose. Consistent input validation and standard error shapes.

Layer	Responsibility
packages/core	ValidationResult, ValidationError, common validators, schema wrapper conventions.
apps	Request schemas, form schemas, business-rule validation.
platform	Optional edge validation through API gateway or API management.

Provider	Recommended adapter package
AWS	@kanbien/platform-aws-api-gateway-validation
Azure	@kanbien/platform-azure-api-management-validation
Open-source/local	@kanbien/platform-oss-validation-zod, @kanbien/platform-oss-validation-ajv

Kanbien v1 recommendation. Use Zod in TypeScript services. Treat gateway validation as extra protection, not the only validation layer.

persistence/

Purpose. Durable state, transactions, pagination, repository conventions, and outbox contracts.

Layer	Responsibility
packages/core	TransactionManager, TransactionContext, PageRequest, Page, repository and outbox interfaces.
apps	Domain repositories, schema migrations, product queries.

Layer	Responsibility
platform	Database client adapters, transaction implementation, outbox storage, connection management.

Provider	Recommended adapter package
AWS	@kanbien/platform-aws-persistence-postgres-rds, @kanbien/platform-aws-persistence-postgres-aurora
Azure	@kanbien/platform-azure-persistence-postgres-flexible-server
Open-source/local	@kanbien/platform-oss-persistence-postgres

Kanbien v1 recommendation. Use PostgreSQL. Keep ORM/client details out of core.

files/

Purpose. Binary objects and file metadata: uploads, exports, generated reports, attachments.

Layer	Responsibility
packages/core	FileStorage, StoredFile, FileObject, content type policy, access contract.
apps	What the file means in a product workflow.
platform	Object storage, signed URLs, scanning pipeline hooks, provider metadata mapping.

Provider	Recommended adapter package
AWS	@kanbien/platform-aws-files-s3
Azure	@kanbien/platform-azure-files-blob
Open-source/local	@kanbien/platform-oss-files-minio

Kanbien v1 recommendation. Use S3 behind FileStorage. Add virus scanning, signed URLs, access logging, retention, and encryption controls when files become sensitive.

events/

Purpose. Facts that something meaningful happened, such as UserCreated or FileUploaded.

Layer	Responsibility
packages/core	DomainEvent, EventEnvelope, EventBus, correlation ID, causation ID, event versioning.
apps	Domain event definitions and business consumers.
platform	Event bus implementation, outbox publisher, schema serialization, replay tooling.

Provider	Recommended adapter package
AWS	@kanbien/platform-aws-events-eventbridge, @kanbien/platform-aws-events-outbox-postgres
Azure	@kanbien/platform-azure-events-event-grid
Open-source/local	@kanbien/platform-oss-events-nats, @kanbien/platform-oss-events-kafka

Kanbien v1 recommendation. Use database outbox plus simple event publishing first. Add EventBridge when routing and independent consumers justify it.

queues/

Purpose. Durable work queues for async processing, retries, and dead-letter handling.

Layer	Responsibility
packages/core	Queue, QueueMessage, QueueHandler, retry metadata, DLQ metadata.
apps	Specific queued work and handlers.
platform	SQS/Service Bus/RabbitMQ/NATS client implementation and worker consumption loop.

Provider	Recommended adapter package
AWS	@kanbien/platform-aws-queues-sqs
Azure	@kanbien/platform-azure-queues-service-bus
Open-source/local	@kanbien/platform-oss-queues-rabbitmq, @kanbien/platform-oss-queues-nats

Kanbien v1 recommendation. Use SQS for AWS-first Kanbien. Require DLQs, idempotency, and queue-depth monitoring.

logging/

Purpose. Structured operational logs used for debugging and incident investigation.

Layer	Responsibility
packages/core	Logger, LoggerFactory, log fields, redaction helpers, correlation context.
apps	Meaningful log messages and product context.
platform	Log transport, JSON formatting, CloudWatch/Azure/Loki/OpenSearch export.

Provider	Recommended adapter package
AWS	@kanbien/platform-aws-logging-cloudwatch, @kanbien/platform-aws-logging-stdout-json
Azure	@kanbien/platform-azure-logging-monitor-logs
Open-source/local	@kanbien/platform-oss-logging-otel-loki

Kanbien v1 recommendation. Use structured JSON logs to stdout plus correlation IDs. Cloud provider log collection can ingest stdout.

monitoring/

Purpose. Health, metrics, alerts, and service-level visibility.

Layer	Responsibility
packages/core	Metrics, HealthCheck, readiness/liveness result, timer helpers.
apps	Business-flow metrics and service health checks.
platform	Metrics exporter, alerting integration, tracing setup, dashboards.

Provider	Recommended adapter package
AWS	@kanbien/platform-aws-monitoring-cloudwatch
Azure	@kanbien/platform-azure-monitoring-application-insights
Open-source/local	@kanbien/platform-oss-monitoring-prometheus, @kanbien/platform-oss-observability-otel

Kanbien v1 recommendation. Expose simple metrics and health checks early. Add SLOs and dashboards as customers depend on the service.

audit/

Purpose. Formal accountability record: who did what, to what, when, where, and under whose authority.

Layer	Responsibility
packages/core	AuditEvent, AuditActor, AuditResource, AuditChange, AuditService.
apps	Which actions are auditable and what changes should be recorded.
platform	Audit storage, archive, export, retention, tamper-evident or immutable mechanisms.

Provider	Recommended adapter package
AWS	@kanbien/platform-aws-audit-postgres, @kanbien/platform-aws-audit-s3-object-lock
Azure	@kanbien/platform-azure-audit-postgres, @kanbien/platform-azure-audit-blob-immutable

Provider	Recommended adapter package
Open-source/local	@kanbien/platform-oss-audit-postgres, @kanbien/platform-oss-audit-pgaudit, @kanbien/platform-oss-audit-immudb

Kanbien v1 recommendation. Use an application audit table from day one for sensitive actions. CloudTrail is useful for AWS account activity, not a substitute for Kanbien application audit.

reporting/

Purpose. Repeatable operational or customer-facing outputs such as CSV, PDF, XLSX, or scheduled report files.

Layer	Responsibility
packages/core	ReportRequest, ReportGenerator, GeneratedReport, report status, report format.
apps	Specific report definitions and queries.
platform	Report file storage, async generation, BI embedding later.

Provider	Recommended adapter package
AWS	@kanbien/platform-aws-reporting-s3, @kanbien/platform-aws-reporting-quicksight
Azure	@kanbien/platform-azure-reporting-blob, @kanbien/platform-azure-reporting-powerbi-embedded
Open-source/local	@kanbien/platform-oss-reporting-superset, @kanbien/platform-oss-reporting-metabase

Kanbien v1 recommendation. Start with app-generated CSV/PDF stored in object storage. Add BI when customers need dashboards or self-service reporting.

analytics/

Purpose. Product and business behaviour telemetry, such as user.created or feature.used.

Layer	Responsibility
packages/core	AnalyticsEvent, AnalyticsIdentity, Analytics, consent and taxonomy concepts.
apps	Event taxonomy and product instrumentation decisions.
platform	Provider/warehouse delivery, batching, consent enforcement, event export.

Provider	Recommended adapter package
AWS	@kanbien/platform-aws-analytics-firehose, @kanbien/platform-aws-analytics-redshift
Azure	@kanbien/platform-azure-analytics-event-hubs, @kanbien/platform-azure-analytics-fabric
Open-source/local	@kanbien/platform-oss-analytics-posthog, @kanbien/platform-oss-analytics-clickhouse

Kanbien v1 recommendation. PostHog is the fastest useful early product analytics option. Use audit separately.

config/

Purpose. Validated environment settings, feature flags, and secret references.

Layer	Responsibility
packages/core	ConfigProvider, config schema, environment name, secret reference types, feature flag interface.
apps	App-specific config keys and feature flag use.
platform	Loading from AppConfig, App Configuration, Parameter Store, Key Vault, Vault, or environment.

Provider	Recommended adapter package
AWS	@kanbien/platform-aws-config-appconfig, @kanbien/platform-aws-config-ssm, @kanbien/platform-aws-secrets-manager
Azure	@kanbien/platform-azure-config-app-configuration, @kanbien/platform-azure-secrets-key-vault
Open-source/local	@kanbien/platform-oss-featureflags-openfeature, @kanbien/platform-oss-featureflags-unleash, @kanbien/platform-oss-secrets-vault

Kanbien v1 recommendation. Create typed startup config validation immediately. Avoid scattered process.env reads.

tenancy/

Purpose. Tenant identity, tenant context, and tenant isolation propagation.

Layer	Responsibility
packages/core	TenantId, TenantContext, TenantResolver, isolation errors, tenant propagation helpers.
apps	Tenant onboarding, tenant settings, tenant admin workflows.
platform	Tenant routing, database selection, row-level security setup, storage/queue partition mapping.

Provider	Recommended adapter package
AWS	@kanbien/platform-aws-tenancy-routing, @kanbien/platform-aws-tenancy-organizations
Azure	@kanbien/platform-azure-tenancy-routing, @kanbien/platform-azure-tenancy-management-groups
Open-source/local	@kanbien/platform-oss-tenancy-postgres-rls, @kanbien/platform-oss-tenancy-kubernetes-namespaces

Kanbien v1 recommendation. Even if single-tenant at launch, put TenantId and TenantContext in the model early.

security/

Purpose. Shared defensive primitives, sensitive data handling, crypto and secret contracts.

Layer	Responsibility
packages/core	Redaction, SensitiveValue, SecureRandom, SecretHasher, CryptoService, rate limit contracts.
apps	Threat-modelled product rules and security-sensitive workflows.
platform	KMS/Key Vault/Vault implementation, WAF/rate-limit integration, secret stores.

Provider	Recommended adapter package
AWS	@kanbien/platform-aws-security-kms, @kanbien/platform-aws-security-waf, @kanbien/platform-aws-secrets-manager
Azure	@kanbien/platform-azure-security-key-vault, @kanbien/platform-azure-security-waf, @kanbien/platform-azure-security-defender
Open-source/local	@kanbien/platform-oss-security-vault, @kanbien/platform-oss-security-opa

Kanbien v1 recommendation. Use redaction and secret interfaces immediately. Do not invent cryptography.

i18n/

Purpose. Internationalization: making the product capable of supporting multiple languages.

Layer	Responsibility
packages/core	Locale, Translator, translation key conventions, message catalog interface, fallback logic.
apps	Product translation strings and notification copy.
platform	Optional machine translation provider for assistive workflows.

Provider	Recommended adapter package
AWS	@kanbien/platform-aws-i18n-translate
Azure	@kanbien/platform-azure-i18n-translator
Open-source/local	@kanbien/platform-oss-i18n-i18next

Kanbien v1 recommendation. Use translation keys and i18next-style catalogs. Machine translation can help draft copy but should not be the source of truth for enterprise UI text.

localization/

Purpose. Locale-specific behaviour: dates, numbers, currency, timezone, address and regional formatting.

Layer	Responsibility
packages/core	LocalizationService, time zone and currency types, date/time formatting, canonical UTC rules.

Layer	Responsibility
apps	User and tenant preferences for locale, timezone, currency.
platform	Mostly standards-based implementation rather than vendor-specific services.

Provider	Recommended adapter package
AWS	No strong AWS-specific default
Azure	No strong Azure-specific default
Open-source/local	@kanbien/platform-oss-localization-intl, @kanbien/platform-oss-localization-formatjs

Kanbien v1 recommendation. Use JavaScript Intl/FormatJS/CLDR-based behaviour. Store timestamps in UTC and money in appropriate minor units.

Part 6 - Events, messaging, and async systems under Option B

Concept map

Concept	Meaning	Core/app/platform split
Domain event	A fact that something meaningful happened, such as UserCreated.	Core defines event envelope; apps define domain event names and payloads.
Event bus	Routes events from publishers to subscribers.	Core defines EventBus; platform implements EventBridge/Event Grid/NATS/etc.
Queue	Stores work until a worker can process it.	Core defines Queue; platform implements SQS/Service Bus/RabbitMQ/etc.
Dead-letter queue	Stores messages that failed after retries.	Core defines DLQ metadata; platform configures DLQs and operations.
Worker	Runtime process that consumes jobs or messages.	Apps own business handlers; platform provides worker runtime helpers.
Async job	A unit of work to run later.	Core defines Job; apps define job type; platform schedules and executes.
Scheduled job	Work created by time rather than a user request.	Core defines Scheduler; platform implements EventBridge Scheduler, Azure Timer, Kubernetes CronJob, etc.

Updated UserCreated flow

```
API
-> creates user inside transaction
-> writes UserCreated to outbox
-> outbox publisher sends event through EventBus
-> event subscriber enqueues welcome email job
-> queue worker sends notification
-> audit records accountability
-> analytics records product signal
```

AWS-first adapter packages for the flow

```
@kanbien/platform-aws-persistence-postgres-rds
@kanbien/platform-aws-events-outbox-postgres
@kanbien/platform-aws-events-eventbridge
@kanbien/platform-aws-queues-sqs
@kanbien/platform-aws-jobs-sqs
@kanbien/platform-aws-notifications-ses
@kanbien/platform-aws-audit-postgres
@kanbien/platform-oss-analytics-posthog
```

Example core event contract

```
export interface DomainEvent<TPayload = unknown> {
type: string;
version: number;
payload: TPayload;
}

export interface EventEnvelope<TPayload = unknown> {
id: string;
event: DomainEvent<TPayload>;
tenantId?: string;
occurredAt: Date;
actorId?: string;
correlationId?: string;
causationId?: string;
}

export interface EventBus {
publish<TPayload>(event: EventEnvelope<TPayload>): Promise<void>;
}
```

Example app usage

```
await eventBus.publish({
id: ids.event(),
event: {
type: "UserCreated",
version: 1,
payload: { userId: user.id, email: user.email },
},
tenantId,
actorId: principal.subject,
occurredAt: clock.now(),
correlationId: requestContext.correlationId,
});
```

Enterprise guardrails

- Use an outbox pattern for events emitted from database transactions.
- Version event payloads from day one.
- Assume at-least-once delivery and design handlers to be idempotent.
- Track correlationId and causationId through API, events, queues, workers, logs, audit, and analytics.
- Create DLQs for every production queue and define operational runbooks for replay or discard.
- Keep commands and events separate: commands ask for action; events state that something happened.

Part 7 - Reporting, analytics, audit, observability, monitoring, and logging

Concept	Purpose	Audience	Data source	Ownership
Reporting	Structured outputs and exports	Customers, ops, finance	Application data, read models, warehouse	Product and operations
Analytics	Understand product behaviour	Product, growth, leadership	Tracked product events	Product/data
Business intelligence	Strategic cross-business analysis	Executives, finance, data	Warehouse/lake plus CRM/billing/support	Data/finance/ops
Audit	Formal accountability evidence	Security, compliance, customers, auditors	Sensitive application actions	Security/compliance/platform
Observability	Understand system behaviour	Engineering/SRE	Logs, metrics, traces	Engineering/platform
Monitoring	Detect health problems	Engineering/SRE/ops	Metrics and health checks	Engineering/platform
Logging	Detailed operational execution record	Developers and SRE	Runtime logs	Engineering

Adapter implications

- Do not implement audit by sending analytics events to PostHog. Audit has stronger integrity, retention, access-control, and export requirements.
- Do not implement monitoring by searching logs manually. Monitoring should have explicit metrics, alerts, and health checks.
- Do not implement reporting by giving customers raw database access. Reporting needs access control, tenant filtering, export audit, and data freshness controls.
- Use separate platform packages for audit storage, analytics delivery, logs, metrics, and reporting exports even if they initially share PostgreSQL or object storage underneath.

```
platform/aws/audit-postgres
platform/aws/audit-s3-object-lock
platform/aws/logging-cloudwatch
platform/aws/monitoring-cloudwatch
platform/oss/analytics-posthog
platform/aws/reporting-s3
```

Part 8 - Multi-tenancy and provider strategy

What multi-tenancy means for Kanbien

Multi-tenancy means one platform serves multiple customer organizations while keeping their data, permissions, events, files, logs, audit records, analytics, jobs, and notifications isolated.

Isolation models

Model	Pros	Cons	When to use
Shared database, shared schema	Cheapest, simplest, fastest early	Highest query discipline required	Default v1 for many SaaS products
Shared database, separate schemas	Better logical isolation	Migration complexity	Mid-market tenants with moderate isolation needs
Separate database per tenant	Strong isolation and restore story	More operations and cost	Large enterprise tenants
Separate deployment per tenant	Maximum isolation	Highest cost and drift risk	Regulated or very large enterprise customers

Core versus platform

```
packages/core/tenancy
TenantId
TenantContext
TenantResolver
TenantIsolationError

platform/aws/tenancy-routing
Maps tenant to AWS region/resources/databases.

platform/oss/tenancy-postgres-rls
Applies PostgreSQL row-level security conventions.

apps/admin/tenant-onboarding
Owns tenant creation workflow and tenant settings.
```

Enterprise considerations

- TenantId must be present in audit, logs, events, queues, files, analytics, reports, and database records where relevant.
- Do not trust tenant ID from a request body. Resolve tenant from authenticated context, route, host, and membership checks.
- Large tenants may need dedicated queues, databases, object storage prefixes/buckets, encryption keys, or regions.
- Tenant fairness matters: background jobs for one tenant should not starve others.

Part 9 - Internationalization and localization ownership

Distinction

Internationalization means designing the product so it can support multiple languages and regions. Localization means adapting the product for a specific language, country, or region.

Responsibility	packages/core	packages/design-system	apps
Locale type	Owns	Uses	Uses
Translator interface	Owns	Uses	Uses
Product translation strings	No	Only for design-system primitives	Owns
Date/currency formatting contract	Owns	Renders	Uses
User locale preference	Defines types	Displays controls	Owns workflow
Notification localization	Defines mechanism	No	Owns templates
RTL layout	No	Owns UI behaviour	Uses

Provider guidance

- Use i18next or an equivalent catalog-driven library for product translations.
- Use Amazon Translate or Azure Translator as assistive translation tools only. Do not make raw machine translation the authoritative source for enterprise UI copy.
- Use JavaScript Intl, FormatJS, and CLDR-backed standards for locale-sensitive date, number, currency, and pluralization behaviour.
- Store timestamps in UTC. Render in the user's or tenant's timezone.
- Store money carefully, usually in minor units plus a currency code, and do not assume every currency has two decimal places.

Part 10 - Building Kanbien v1 with the updated architecture

Must have on day one

Capability	Core module	Initial platform package	Why
Typed config	config	platform/aws/config-env or config-appconfig later	Prevents scattered environment reads and startup surprises.
Structured logging	logging	platform/aws/logging-stdout-json	Enables debugging, incident response, and correlation.
Validation	validation	platform/oss/validation-zod	Standard API errors and safer input handling.
Authentication	authn	platform/aws/authn-cognito	Consistent identity model.
Authorization	authz	platform/local/authz-simple-rbac initially	Permissions are painful to retrofit.
Tenancy	tenancy	platform/local/tenancy-single-tenant initially	TenantId and TenantContext should exist early.
Persistence	persistence	platform/aws/persistence-postgres-rds	Transactions, pagination, and outbox patterns.
Audit	audit	platform/aws/audit-postgres	Enterprise trust and accountability.
Files	files	platform/aws/files-s3 when needed	Secure file handling behind FileStorage.
Queues/jobs	queues, async-jobs	platform/aws/queues-sqs, jobs-sqs when needed	Async work, retries, and DLQs.

Should have as the product grows

- EventBridge adapter when event routing beyond the app/worker boundary becomes useful.
- SES notification adapter when email types, templates, delivery status, and preferences grow.
- CloudWatch monitoring adapter when production users depend on the system.
- PostHog analytics adapter when product decisions need activation, adoption, and retention data.
- Reporting export packages when customers need CSV/PDF/XLSX exports and scheduled reports.
- i18n/localization packages before international customers and localized notifications become urgent.

Enterprise future

- Fine-grained authorization with Verified Permissions, OpenFGA, or OPA.
- Dedicated tenant isolation patterns: separate database, separate bucket, separate queue, separate region, or separate deployment.
- Immutable audit archive with S3 Object Lock or Azure immutable Blob where required.
- Warehouse-backed analytics and BI with Redshift, Fabric, ClickHouse, Superset, Metabase, Power BI, or QuickSight.
- SCIM, SAML, SSO, MFA, customer-managed roles, and delegated administration.
- Event schema governance, replay tooling, and cross-region data residency controls.

Part 11 - Final architecture reference

Final packages/core structure

```
packages/core/
package.json
README.md
src/
index.ts
shared/
config/
logging/
monitoring/
security/
authn/
authz/
tenancy/
validation/
persistence/
events/
queues/
async-jobs/
notifications/
files/
audit/
reporting/
analytics/
i18n/
localization/
testing/
```

Final platform structure using Option B

```
platform/
shared/
package.json
src/
retry/
serialization/
observability/
idempotency/
adapter-errors/

local/
events-in-memory/
queues-in-memory/
files-local/
notifications-console/
authn-fake/
authz-simple-rbac/
analytics-noop/

aws/
authn-cognito/
authz-verified-permissions/
config-appconfig/
config-ssm/
secrets-manager/
security-kms/
files-s3/
queues-sqs/
events-eventbridge/
events-outbox-postgres/
jobs-sqs/
scheduler-eventbridge/
notifications-ses/
persistence-postgres-rds/
persistence-postgres-aurora/
audit-postgres/
audit-s3-object-lock/
logging-stdout-json/
logging-cloudwatch/
monitoring-cloudwatch/
analytics-firehose/
reporting-s3/

azure/ # future, when real implementation is needed
oss/ # future, when real implementation is needed
```

Relationship to infra

```
infra/aws creates:
S3 bucket
SQS queue and DLQ
EventBridge bus
RDS/Aurora database
IAM roles
KMS keys
CloudWatch alarms

platform/aws uses those resources at runtime.
packages/core defines what those resources mean to Kanbien.
```

Repository guardrails

- No AWS SDK imports outside platform/aws/* packages.
- No Azure SDK imports outside platform/azure/* packages.
- No vendor SDK imports inside packages/core.
- Feature code imports @kanbien/core types and receives dependencies through explicit composition.
- Only app bootstrap/composition code imports platform adapter packages.
- Every adapter package should have contract tests against the corresponding core port.

- Every provider adapter README should explain guarantees, limitations, required infra, config keys, and operational concerns.

Most common founder mistakes

- Overbuilding platform machinery before product risk is resolved.
- Underinvesting in authz, tenancy, audit, validation, and logging because they feel like later enterprise work.
- Confusing code reuse with architecture and moving unstable feature logic into core.
- Building for a hypothetical enterprise buyer rather than likely near-term customers.

Most common engineer mistakes

- Creating generic frameworks too early.
- Letting vendor SDKs leak into core or feature code.
- Treating logs, audit, analytics, reporting, and monitoring as interchangeable.
- Ignoring versioning for events, APIs, schemas, and authorization contracts.
- Making everything globally available instead of passing dependencies explicitly.

Ten-year principles

- Core says what; platform says how; infra provides where; apps decide why and when.
- Keep dependency direction clean even when shortcuts seem faster.
- Prefer boring, explicit, testable architecture over fashionable patterns.
- Start simple, but leave clear seams for future replacement.
- Treat tenant isolation and audit as enterprise trust foundations, not optional add-ons.
- Do not put product workflows in core.
- Make important concepts visible in types: TenantId, Principal, Permission, AuditEvent, EventEnvelope, Locale, Currency, CorrelationId.
- Design for operations: running, observing, debugging, securing, recovering, auditing, and evolving the system matters as much as writing code.

Appendix A - Selected vendor and standards source notes

The provider recommendations in this guide were checked against official or primary documentation where possible. These notes are not procurement advice; re-check service limits, pricing, regional availability, and deprecations before committing to an implementation.

S1 Amazon Cognito user pools - https://docs.aws.amazon.com/cognito/latest/developerguide/cognito-user-pools.html

S2 Microsoft Entra External ID - https://learn.microsoft.com/en-us/entra/external-id/customers/

S3 Keycloak - https://www.keycloak.org/

S4 Amazon Verified Permissions - https://docs.aws.amazon.com/verifiedpermissions/latest/userguide/what-is-avp.html

S5 Microsoft Entra External ID app roles - https://learn.microsoft.com/en-us/entra/external-id/customers/how-to-use-app-roles-customers

S6 OpenFGA - https://openfga.dev/

S7 Open Policy Agent - https://openpolicyagent.org/docs

S8 Amazon Aurora PostgreSQL - https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/Aurora.AuroraPostgreSQL.html

S9 Azure Database for PostgreSQL - https://learn.microsoft.com/en-us/azure/postgresql/

S10 PostgreSQL - https://www.postgresql.org/

S11 Amazon S3 - https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html

S12 Azure Blob Storage - https://learn.microsoft.com/en-us/azure/storage/blobs/storage-blobs-introduction

S13 MinIO S3 API compatibility - https://docs.min.io/aistor/developers/s3-api-compatibility/

S14 Amazon EventBridge - https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-what-is.html

S15 Azure Event Grid - https://learn.microsoft.com/en-us/azure/event-grid/overview

S16 NATS JetStream - https://docs.nats.io/nats-concepts/jetstream

S17 Apache Kafka - https://kafka.apache.org/

S18 Amazon SQS - https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/welcome.html

S19 Azure Service Bus - https://learn.microsoft.com/en-us/azure/service-bus-messaging/service-bus-queues-topics-subscriptions

S20 RabbitMQ - https://www.rabbitmq.com/

S21 AWS Step Functions - https://docs.aws.amazon.com/step-functions/

S22 Amazon EventBridge Scheduler - https://docs.aws.amazon.com/scheduler/latest/UserGuide/what-is-scheduler.html

S23 Azure Durable Functions - https://learn.microsoft.com/en-us/azure/durable-task/durable-functions/durable-functions-overview

S24 Temporal - https://docs.temporal.io/

S25 Kubernetes CronJob - https://kubernetes.io/docs/concepts/workloads/controllers/cron-jobs/

S26 Amazon SES - https://docs.aws.amazon.com/ses/

S27 Azure Communication Services - https://learn.microsoft.com/en-us/azure/communication-services/

S28 Novu - https://docs.novu.co/platform

S29 Amazon Pinpoint end of support notice - https://docs.aws.amazon.com/pinpoint/latest/userguide/migrate.html

S30 Zod - https://zod.dev/

S31 Ajv - https://ajv.js.org/

S32 API Gateway request validation - https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-method-request-validation.html

S33 Azure API Management validate-content - https://learn.microsoft.com/en-us/azure/api-management/validate-content-policy

S34 AWS AppConfig - https://docs.aws.amazon.com/appconfig/latest/userguide/what-is-appconfig.html

S35 Azure App Configuration - https://learn.microsoft.com/en-us/azure/azure-app-configuration/overview

S36 OpenFeature - https://openfeature.dev/community/mission-vision/

S37 Unleash - https://docs.getunleash.io/

S38 Flagsmith - https://docs.flagsmith.com/

S39 AWS Secrets Manager - https://docs.aws.amazon.com/secretsmanager/latest/userguide/intro.html

S40 AWS KMS - https://docs.aws.amazon.com/kms/latest/developerguide/overview.html

S41 Azure Key Vault - https://learn.microsoft.com/en-us/azure/key-vault/

S42 HashiCorp Vault - https://developer.hashicorp.com/vault/docs

S43 AWS WAF - https://docs.aws.amazon.com/waf/

S44 Azure Web Application Firewall - https://learn.microsoft.com/en-us/azure/web-application-firewall/

S45 Amazon CloudWatch - https://docs.aws.amazon.com/cloudwatch/

S46 Azure Monitor - https://learn.microsoft.com/en-us/azure/azure-monitor/

S47 OpenTelemetry - https://opentelemetry.io/docs/

S48 Grafana Loki - https://grafana.com/docs/loki/latest/

S49 Prometheus - https://prometheus.io/

S50 AWS CloudTrail - https://docs.aws.amazon.com/awscloudtrail/latest/userguide/cloudtrail-user-guide.html

S51 Amazon S3 Object Lock - https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lock.html

S52 Azure Activity Log - https://learn.microsoft.com/en-us/azure/azure-monitor/platform/activity-log

S53 Azure immutable Blob storage - https://learn.microsoft.com/en-us/azure/storage/blobs/immutable-storage-overview

S54 pgAudit - https://pgaudit.org/

S55 immudb - https://docs.immudb.io/

S56 Amazon QuickSight pixel-perfect reports - https://docs.aws.amazon.com/quick/latest/userguide/working-with-reports.html

S57 Power BI Embedded - https://learn.microsoft.com/en-us/power-bi/developer/embedded/

S58 Apache Superset - https://superset.apache.org/

S59 Metabase - https://www.metabase.com/docs/latest/

S60 Amazon Data Firehose - https://docs.aws.amazon.com/firehose/latest/dev/what-is-this-service.html

S61 Amazon Redshift - https://docs.aws.amazon.com/redshift/latest/mgmt/welcome.html

S62 Azure Event Hubs - https://learn.microsoft.com/en-us/azure/event-hubs/event-hubs-about

S63 Microsoft Fabric - https://learn.microsoft.com/en-us/fabric/fundamentals/microsoft-fabric-overview

S64 PostHog - https://posthog.com/docs

S65 ClickHouse - https://clickhouse.com/docs/intro

S66 Amazon Translate - https://docs.aws.amazon.com/translate/latest/dg/what-is.html

S67 Azure Translator - https://learn.microsoft.com/en-us/azure/ai-services/translator/

S68 i18next - https://www.i18next.com/

S69 Unicode CLDR - https://cldr.unicode.org/

S70 JavaScript Intl - https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl

S71 FormatJS - https://formatjs.io/

S72 AWS Organizations - https://docs.aws.amazon.com/organizations/latest/userguide/orgs_introduction.html

S73 Azure management groups - https://learn.microsoft.com/en-us/azure/governance/management-groups/overview

S74 PostgreSQL Row-Level Security - https://www.postgresql.org/docs/current/ddl-rowsecurity.html

S75 Postal open-source mail delivery platform - https://docs.postalserver.io/